QueryAPIGate HOW-TO GUIDE 32 · OPERATE

Deploy with Docker for real

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=32-deploy-with-docker

---

Time: 30 minutes. **You'll end up with:** QueryAPIGate running in Docker Compose behind HTTPS, with its data on a persistent volume, secrets in an env file, a health check, and the commands for backups and upgrades.

The repository's own `docker-compose.yml` is a **one-command demo** - a demo key, a throwaway PostgreSQL, localhost only. Don't deploy that. This guide builds the production shape that DEPLOYMENT.md describes in full, step by step. It was run end to end, with Caddy using a locally issued certificate on `https://localhost:8443` instead of a public domain.

Step 1: A folder with three files

```text
queryapigate/
├── docker-compose.yml
├── Caddyfile
└── .env
```

`docker-compose.yml`:

```yaml
services:
  queryapigate:
    image: ghcr.io/anantharajuc/queryapigate:0.48.0   # pin a version - upgrades should be deliberate
    restart: unless-stopped
    env_file: .env
    environment:
      QUERYAPIGATE_TRUST_PROXY: "1"                   # one proxy in front: Caddy
    volumes:
      - queryapigate-data:/data                       # queryapigate.db lives here
    networks:
      - internal

  proxy:
    image: caddy:2
    restart: unless-stopped
    ports:
      - "443:443"
      - "80:80"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy-data:/data                              # certificates
    networks:
      - internal
    depends_on:
      - queryapigate

networks:
  internal:

volumes:
  queryapigate-data:
  caddy-data:
```

Only the proxy publishes ports; QueryAPIGate is reachable only through it.

`Caddyfile`:

```caddyfile
api.example.com {
    reverse_proxy queryapigate:5000
}
```

Caddy gets a certificate for `api.example.com` by itself, provided DNS points at this host and ports 80 and 443 are open. More on proxies: Put QueryAPIGate behind a reverse proxy.

`.env` - secrets, never committed:

```shell
QUERYAPIGATE_API_KEY=<openssl rand -hex 32>
QUERYAPIGATE_SECRET_KEY=<a Fernet key - see guide 22>
WAREHOUSE_PASSWORD=<your database's password>
```

```shell
chmod 600 .env
```

`QUERYAPIGATE_API_KEY` is the admin key: full control, so make it long and random and give it to few people. It isn't optional in a container: with no key (or administrator, JWT or MCP OAuth) the server runs in open-access mode, which answers only local clients - and from inside a container the caller's address is the Docker or pod network, not loopback, so every real request is refused `403 open_access_remote` until you set one. `QUERYAPIGATE_SECRET_KEY` encrypts connection passwords in the store (guide 22); keep a copy in your secret manager.

Step 2: Start it

```shell
docker compose run --rm queryapigate queryapigate init     # optional: inactive template connections
docker compose up -d
docker compose ps                                           # queryapigate should become "healthy"
curl https://api.example.com/health
# {"status": "ok", "version": "...", "time_zone": "Etc/UTC", "utc_offset": "+00:00"}
```

Verified: the container reported `healthy` (the image checks `/health` every 30 seconds), and `/health` answered over HTTPS through Caddy. `init` is optional - it adds one inactive template connection per database type; you can also run it after `up` with `docker compose exec`, and the server picks them up at once.

The image already runs as a non-root user, under gunicorn with one worker and eight threads - nothing to configure.

Step 3: Add your database and a key

Open `https://api.example.com/console`, enter the admin key, and add a connection under **Connections** - with the password as `${WAREHOUSE_PASSWORD}`, or typed in (it's encrypted, since `QUERYAPIGATE_SECRET_KEY` is set). Then create scoped keys for whatever will call the API (guide 13) - the admin key shouldn't leave your team.

Every driver and optional feature is in the image: PostgreSQL, MySQL, ClickHouse, DuckDB, SQLite, MongoDB, `allowed_tables`, JWT, Redis caching and `queryapigate mcp`. Verified in the container: a query on a DuckDB file in `/data`, a table-restricted key refused with `table_not_allowed`, and `queryapigate mcp` starting. For H2 or a generic JDBC driver, use the `-h2` tag, which adds Java.

One thing that catches people: a SQLite or DuckDB connection's file must already exist, inside the container - put it on the `/data` volume (a relative path is resolved against `/data`).

Step 4: Day two

Logs: `docker compose logs -f queryapigate` - one line per request, with the request id and key name (guide 36).

Backups - verified in the container:

```shell
docker compose exec queryapigate queryapigate backup /data/backup.db --force
docker compose cp queryapigate:/data/backup.db "./backups/queryapigate-$(date +%F).db"
docker compose exec queryapigate rm /data/backup.db
```

Schedule it from the host's cron, and copy the result off the machine. Restoring: guide 33.

Upgrades: read the changelog, back up, change the pinned tag, `docker compose pull && docker compose up -d`. The store is upgraded on first start. See guide 34.

Monitoring: `/metrics` for Prometheus; don't expose it publicly (guide 35).

Optional services

Add them to the same Compose file when you need them - the same image, different commands. Details in DEPLOYMENT.md:

SERVICE	WHEN	COMMAND
`redis` (`redis:7-alpine`) + `QUERYAPIGATE_REDIS_URL`	cached responses that survive restarts	-
`events`	live events for many clients	`queryapigate events --host 0.0.0.0`
`mcp`	AI agents over MCP	`queryapigate mcp --host 0.0.0.0`

Give each the same `env_file` and volume (or `QUERYAPIGATE_DATABASE_URL`), so they share the store.

When one instance isn't enough

This guide runs one instance - with SQLite on a volume, or PostgreSQL, that's fully supported. For more capacity or availability, run several: they need a PostgreSQL store and Redis, and then share everything. Don't just raise the replica count of this Compose file - each would get its own SQLite volume and be a separate server. See Run several instances, and keep one gunicorn worker per container either way (Scaling out).

Checklist before going live

- ☐ The image tag is a pinned version, not `latest`.
- ☐ `.env` is `chmod 600`, not in version control, and backed up in your secret manager.
- ☐ Only the proxy publishes ports; `QUERYAPIGATE_TRUST_PROXY` matches the number of proxies.
- ☐ `/metrics` is restricted at the proxy.
- ☐ Scoped keys for every caller; the admin key stays with your team.
- ☐ A backup job runs, and you've restored one once.
- ☐ Read the security hardening checklist.

Next steps

- Back up and restore the store.
- Wire up Prometheus and Grafana.
- Upgrade to a new version safely.

READ NEXT

Back up and restore the store

QUESTIONS OR PROBLEMS

Discord · GitHub issues

LATEST VERSION OF THIS PAGE

anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=32-deploy-with-docker

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit

@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate